Høgskulen på Vestlandet

DAT107 - Databaser

Video 09: Via Objekter til NoSQL

Introduksjon til NoSQL

Dag Larsen
Oppdatert av Lasse Jenssen

Læringsmål

- Forstå begrensningene ved relasjonsdatabaser og motivasjonen for objektrelasjonelle databaser og NoSQL-databaser.
- Kunne lage brukerdefinerte datatyper i den objektrelasjonelle databasen PostgreSQL.

Litteratur

NoSQL

Læreboka:

- ✓ Kapittel 15
  - 15.1 – Motivasjon
  - 15.2 – Objekter og databaser
  - 15.3 – NoSQL-landskapet

Høgskulen på Vestlandet

Motivasjon

Veien mot NoSQL

Kritikk av relasjonsdatabaser

Veien via objekter til NoSQL databaser

Høgskulen på Vestlandet

Relasjonsdatabaser egner seg mindre godt for å håndtere komplekse datastrukturer

- ✓ Geografiske InformasjonsSystemer (GIS)
- ✓ Tekniske tegninger (DAK/DAP)
- ✓ Multimedia
- ✓ Semistrukturerte dokumenter
- ✓ Sosiale medier
- ✓ Stordata (BigData) samlet inn på «tingenes internett»

Bruker GIS som eksempel

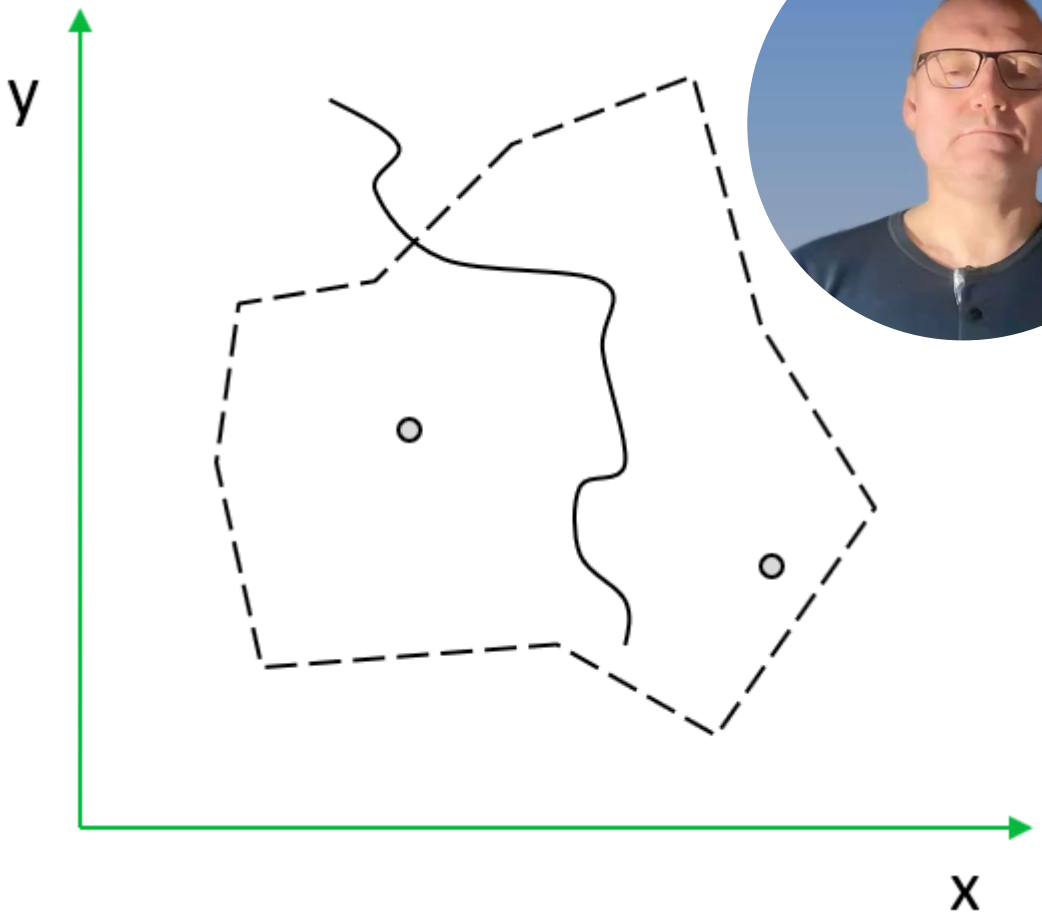
- ✓ Problemet med å representere digitale kart i tabeller (relasjonsdatabaser)

GIS som eksempel

Veien via objekter til NoSQL databaser

Høgskulen på Vestlandet

- ✓ Et kart på vektorform består av punkter, linjer og flater.
- ✓ Et punkt kan representere en by, en polylinje en vei, og et polygon et fylke.
- ✓ Et punkt kan representeres som (X, Y).
  Mulig tabell: **Punkt(X, Y)**
- ✓ Et linjestykke kan representeres ved startpunkt og endepunkt.
  Mulig tabell: **Linjestykke(X1, Y1, X2, Y2)**
- ✓ Fristende å lagre punkter som verdier:
  Tenkt tabell: ~~Linjestykke(P1, P2)~~



Men … Har ikke en datatype Punkt

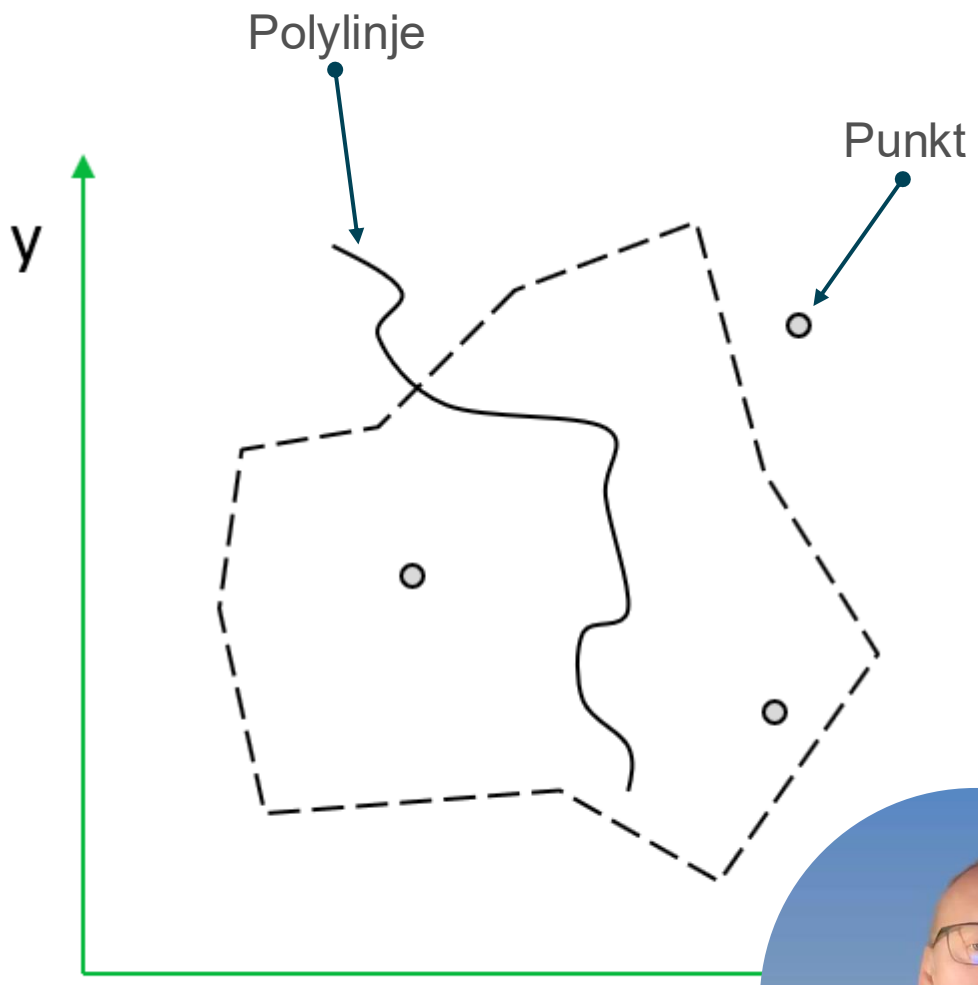
GIS som eksempel

Veien via objekter til NoSQL databaser

- ✓ En **polylinje** kan bestå av mange punkter.
- ✓ Et **punkt** kan inngå i mange polylinjer.

Vi kan tenke oss følgende **tabeller**...

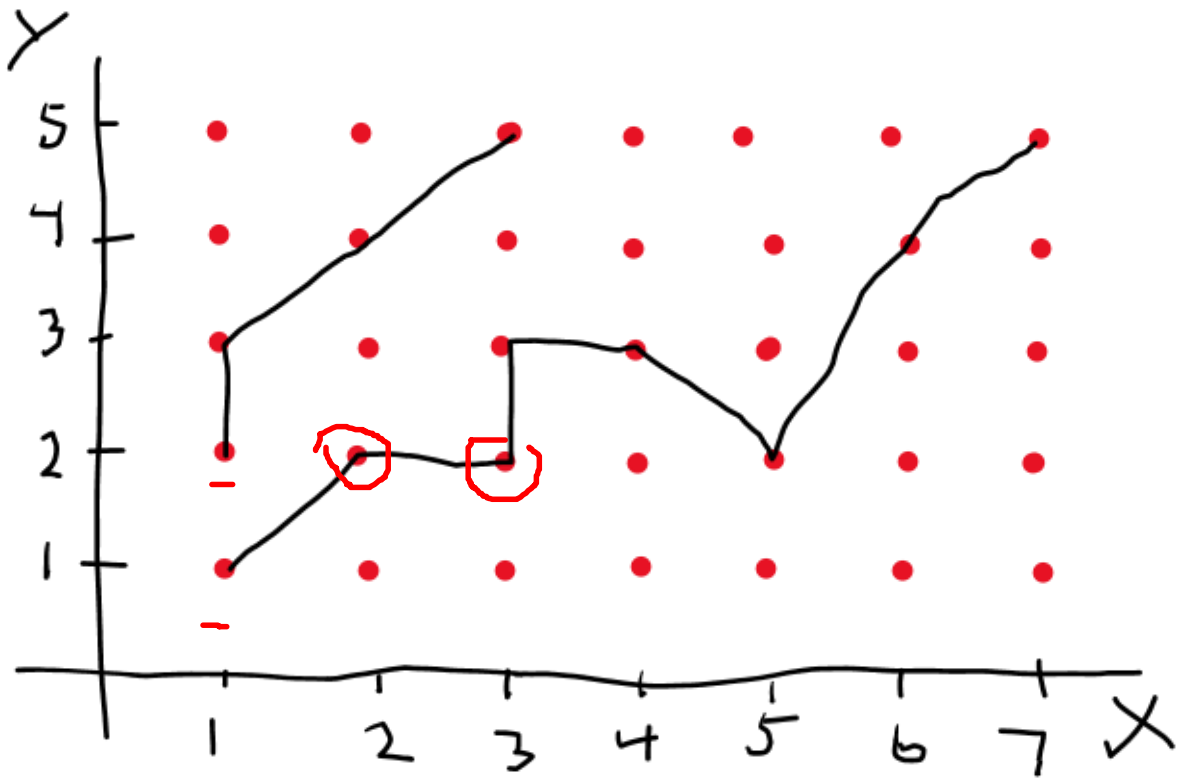
- Punkt(Pid, X, Y,...)
- Linje(Lid, ...)
- PunktLinje(Lid*, Pid*, SekvensNr) (Koblingstabell)



NB! Over har vi tildelt punkter og linjer et løpenummer: «Pid» og «Lid»

Normalisert datamodell for kartdata

Polylinje



Linje

Lid	...
1	...

Lid	Pid	Sekv
1	1	1
1	7	2
1	12	3
1	13	4
1	18	5
1	22	6
1	29	7
1	35	8
2	2	1
2	3	2
2	9	3
2	15	4

Punkt

Pid	X	Y
1	1	1
2	1	2
3	1	3
4	1	4
5	1	5
6	2	1
7	2	2
8	2	3
9	2	4
10	2	5
11	3	
12	3	
13		
14		
15		
16		
...		

Normalisert datamodell for kartdata

Polygon (linje henger sammen)

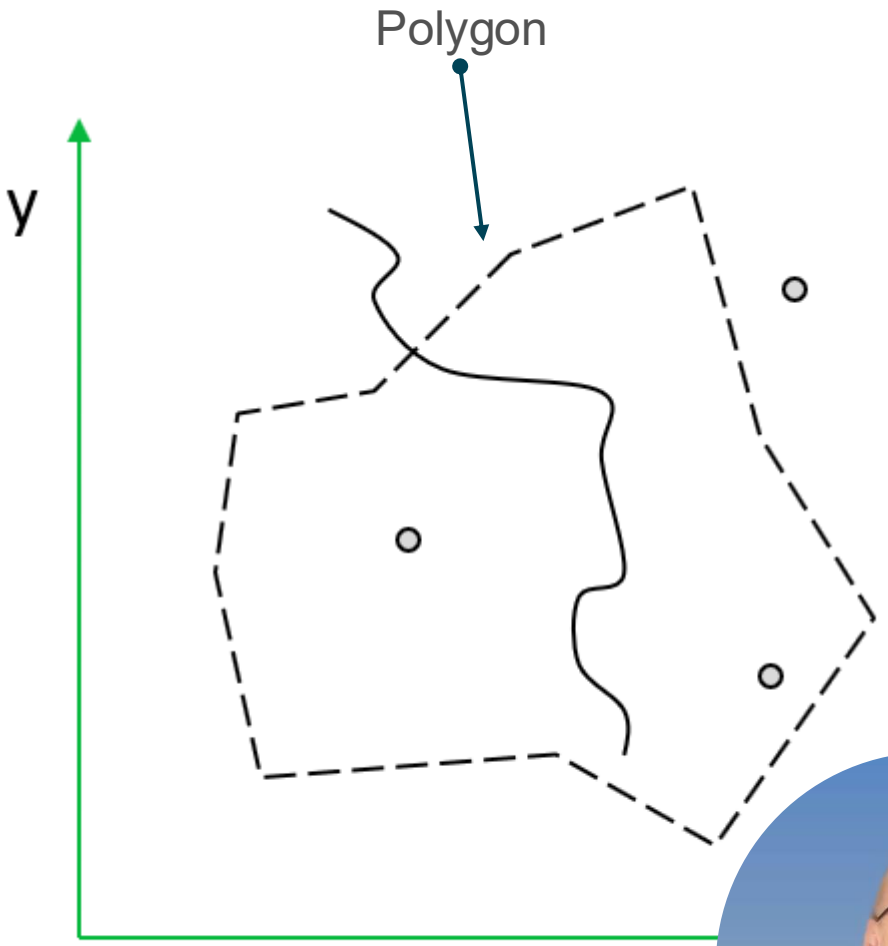
- ✓ Et **polygon** kan representeres på samme måte som polylinjer, der vi krever at **siste punkt har samme koordi**nater som første punkt.

Geometriobjekter **kan** altså representeres i tabeller ved hjelp av enkle datatyper.

Men (enkle) kartoperasjoner er **vanskelige å kode med SQL.**

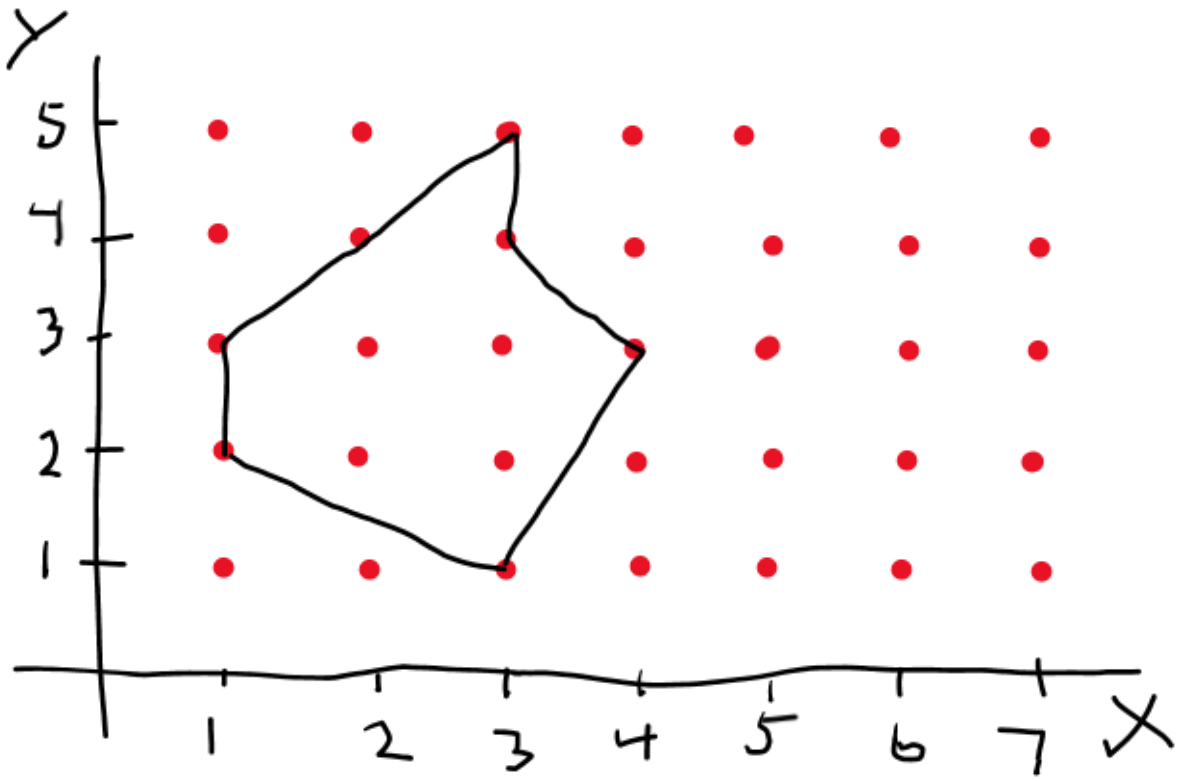
Eksempel:

- ✓ Er et bestemt punkt (et klikk i et kart) innenfor et polygon?
- ✓ Krysser to polylinjer hverandre?



Normalisert datamodell for kartdata

Polygon (linje henger sammen)



Polygon

GId	...
1	...

Gid	Pid	Sekv
1	2	1
1	3	2
1	9	3
1	15	4
1	14	5
1	18	6
1	11	7
1	2	8

Punkt

Pid	X	Y
1	1	1
2	1	2
3	1	3
4	1	4
5	1	5
6	2	1
7	2	2
8	2	3
9	2	4
10	2	5
11	[illegible]	[illegible]
12		
13		
14		
15		
16		
...		

Geometrisk Operasjon: Distanse

Et tenkt eksempel

Høgskulen på Vestlandet

Lag en spørring som finner alle byer som ligger «langs» (500 meter) Glomma.

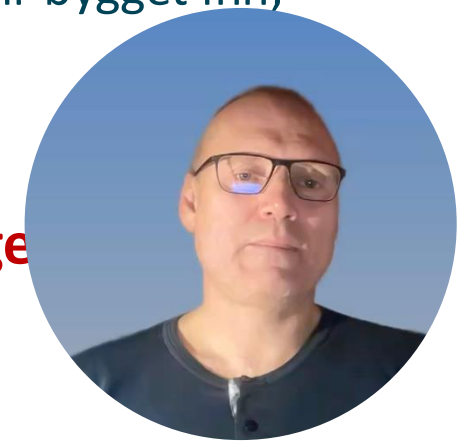
For hver by (med gitte koordinater) må vi beregne **avstanden til polylinjen Glomma**, som betyr at vi må beregne avstanden fra hver by til hvert av linjestykkene som Glomma er bygd opp fra.

Anta:
At vi utvider SQL med datatypene Point og Polyline.

Det vil da være naturlig at en del grunnleggende operasjoner på punkter og linjer også blir bygget inn, f.eks. avstand:

```
SELECT Byer.*
FROM Byer, Elv
WHERE Distance(By.Punkt, Elv) < 0.5
AND Elv.Navn = "Glomma"
```

Hva er poenge



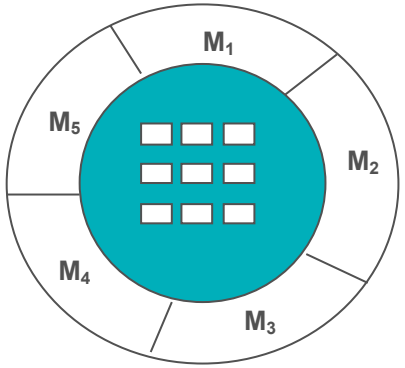
Hva er poenget?

Repetisjon: Objekt = Data + Metoder

Høgskulen på Vestlandet

- ✓ Et **objekt** er en ting, et begrep. *Eksempel: Ansatt*
- ✓ Et objekt er til enhver tid i en bestemt **tilstand**.

AnsattNr=1, Etternavn="Hansen", Lønn= 475.000



- ✓ Et objekt kan gå over i en ny tilstand, men er fremdeles «det samme» objektet.

Hansen er Hansen selv om hun får lønnsøkning.

- ✓ Et objekt har **metoder**.

enAnsatt.endreLønn(nyLønn)

- ✓ Objekter beskytter sine dataelementer (**innkapsling**). All aksess går via metodene.

Hva er poenget?

Repetisjon: Objekt vs. Klasser

Høgskulen på Vestlandet

- ✓ En **klasse** er en «mal» for å lage objekter.

  Fra klassen Ansatt kan vi generere objekter for ansatt Per, ansatt Lise og ansatt Jens.

- ✓ Klassen beskriver hva slags **data** og hvilke **metoder** («handlinger») objektene av klassen skal inneholde.

- ✓ Under programkjøring blir det **generert objekter** fra klassene (**instansiering**).

- ✓ Alle objektene av en gitt klasse har samme struktur.

Hva er poenget?

Objekt databaser vs Objekt-relasjonelle databaser

Høgskulen på Vestlandet

- ✓ På 1990-tallet ble det utviklet **objektdatabaser** for å håndtere komplekse datastrukturer, f.eks. kartdata.

- ✓ Som svar på dette ble relasjonsdatabasene utvidet med objektorienterte mekanismer:
  - Mulighet for å definere egne datatyper med tilhørende metoder, som kan brukes i tabelldefinisjon og SQL.
  - «Collection»-klasser: Tabell, sekvens, multiset.
  - Subklasser (spesialisering)

- ✓ Slike hybridsystemer kalles for **objektrelasjonelle** databaser og fikk **«større kommersielt gjennomslag»** enn de rene objektdatabasene (for denne typen data).

Hva er poenget?

Geometrisk Operasjon: Distanse

Høgskulen på Vestlandet

- ✓ Behandling av kartdata viser **behovet for sammensatte, brukerdefinerte datatyper**.
- ✓ I **objektorienterte databasesystemer og programmeringsspråk** kan man definere sine **egne datatyper med tilhørende operasjoner**.
- ✓ Kartdata er kun et eksempel – men et godt eksempel – på behovet for brukerdefinerte datatyper.
  - Geometridatatyper er beskrevet i standarden ***SQL multimedia and application packages*** (***ISO/IEC 13249 standard***, ofte referert til som ***SQL/MM***).
  - PostgreSQL: PostGIS
  - Oracle: Oracle Spatial
  - MS SQL Server: SQL Server Spatial

Objekter & Databaser

Veien mot NoSQL

Objekt-relasjonell database

PostgreSQL

Høgskulen på Vestlandet

- ✓ Støtte for SQL og standard datatyper. — SQL med Dag
- ✓ Dessuten:
  - Brukerdefinerte datatyper (skrevet i lav-nivå programmerings språk)
  - Array-typer
  - Oppramstyper (Enumerated types)
  - Sammensatte typer (Composite types)
  - Intervalltyper (Range types)
  - Geometrityper (PostGIS)
  - XML- og JSON-datatyper — Sett på tidligere.

Array-typer, Oppramstyper, Sammensatte typer, Intervalltyper: Neste slides

Sammensatte typer (Composite Types)

PostgreSQL: Objekt-relasjonell database

Høgskulen på Vestlandet

```
CREATE TYPE AdresseType AS (
  Gate   VARCHAR(50),
  Nummer INTEGER
);
```

```
CREATE TABLE student (
  id      integer primary key,
  fornavn varchar(25),
  adresse AdresseType
);
```

«*Kan*» brukes som datatype for en kolonne Adresse i en persontabell (Se figur)

Men IKKE vanlig, og sjeldent hensiktsmessig

Id	Fornavn	Adresse
5	Ole	(Furulia, 28)

Sammensatte typer (Composite Types)

PostgreSQL: Objekt-relasjonell database

Høgskulen på Vestlandet

Vi bruker funksjonen **ROW** for å bygge adressen “Furulia 28” fra et gatenavn og et gatenummer:

```
INSERT INTO Student(Id, Fornavn, Adresse)
     VALUES (5, 'Ole', ROW('Furulia', 28));
```

Og bruker **prikknotasjon** for å hente ut igjen komponentene fra slike sammensatte verdier:

```
SELECT Id, Fornavn, (Adresse).Gate
FROM Student;
```

Arrays & Oppramstyper (Enum)

PostgreSQL: Objekt-relasjonell database

Høgskulen på Vestlandet

- ✓ En **array** brukes for å ta vare på en liste med verdier av samme datatype, f.eks. en liste med telefonnumre.

  Eksempel på kolonnedefinisjon: `tlf varchar(8)[]`
  For å få tak i første verdi: `tlf[0]`

- ✓ En **oppramstype (Enum)** blir definert ved å liste opp alle verdiene:

```
CREATE TYPE StatusType AS ENUM ('aktiv', 'permisjon', 'sluttet');
```

Mer vanlig med oppslagstabell (for kolonne datatype)

Id	Fornavn	Adresse	Tlf	Status
5	Ole	(Furulia, 28)	{'22334455','88776655'}	aktiv

Eksempel

Sammesatte-, Arrays- & Oppramstyper

```
CREATE TABLE student(
   id          integer primary key,
   fornavn     varchar(25),
   adresse     AdresseType,                    -- Sammensatt (Composite) type
   tlf         char(8)[],                      -- Array type
   status      StatusType                      -- Opprams (ENUM) type
);

INSERT INTO Student(Id, Fornavn, Adresse, tlf, status)
   VALUES (5, 'Ole', ROW('Furulia', 28), ARRAY['22334455','88776655'], 'aktiv');
```

Eller …

```
INSERT INTO Student(Id, Fornavn, Adresse, tlf, status)
   VALUES (… ,{"22334455","88776655"}', 'aktiv');

SELECT * FROM student;
```

Når brukes hva?

Oppsummering

Høgskulen på Vestlandet

- ✓ PostgreSQL støtter i tillegg til relasjonsdatabase: egendefinerte datatyper/ oppramstyper/array-typer, dokumentdatabase (både XML og JSON) samt verdi/nøkkeldatabase
- ✓ Forvirrende? Når skal man bruke hva?
  - **"Vanlige" datatyper**: Så lenge du ikke har en svært god grunn for å gjøre noe annet.
  - **Array-typer**: For bruk i SQL prosedyre språk (PL/pgSQL)
  - **Sammensatte typer**: For bruk i SQL prosedyre språk (PL/pgSQL)
  - **Oppramstyper (ENUM)**: Bruk oppslagstabell (med referanseintegritet) i stedet
  - **XML/JSON**: semistrukturerte data, og/eller hvis det ikke er behov for å normalisere data (hybrid løsning)
- ✓ Attributter lagret i hstore/xml/json/array/brukerdefinerte typer kan ikke nødvendigvis indekseres
- ✓ Husk at dette bryter med relasjonsmodellen!

Oppsummering

Database historien

Høgskulen på Vestlandet

1960-tallet: hierarkiske databaser og nettverksdatabaser

1970-tallet:

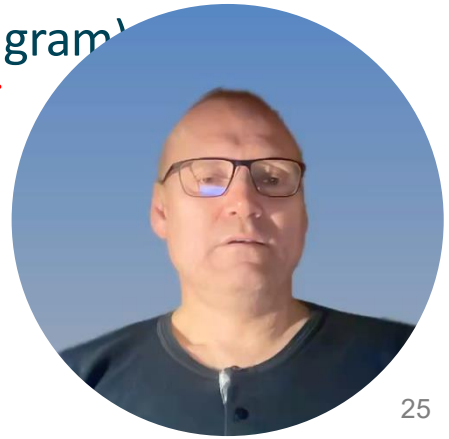
- ✓ Edgar F. Codd [Codd 1970] argumenterte mot denne måten å organisere data på (se kap. 6), fordi den var basert på navigasjonsstrukturer, eller bruk av «pekere».
- ✓ Relasjonsdatabaser

1990-tallet:

- ✓ Objekt-relasjonsdatabaser: Utvidelser av relasjonsdatabasene

2000-tallet/ 2010-tallet:
(med store internasjonale selskaper og sosiale mediaer: Amazon, Google, Facebook, Instagram)

- ✓ NoSQL databasene gjør inn-tog.



2000/2010-tallet: NoSQL databaser

Via objekter til NoSQL

Høgskulen på Vestlandet

1980s: Commercialization of the RDBMS: Oracle, DB2, MS SQL Server

1990s: Internett. Client-Server Architecture and ORDBMS

2000s: Smartphones. E-Business.

1970s: Codd: Relational Datamodel and SQL

2010s: Big Data. NoSQL databases.

2020s: Multi-model DB

1960 1970 1980 1990 2000 2010 2020

'01 JSON

'98 XML 1.0

'12 Amazon DynamoDB

'68 IBM IMS (Hierachical DB)

'79 Oracle DB

'08 Cassandra

'09 MongoDB

'89 Microsoft SQL Server

'10 Neo4J

Mid-90s: MySQL & PostgreSQL

'07 iPhone

'73 IBM IDMS (Network-model DB)

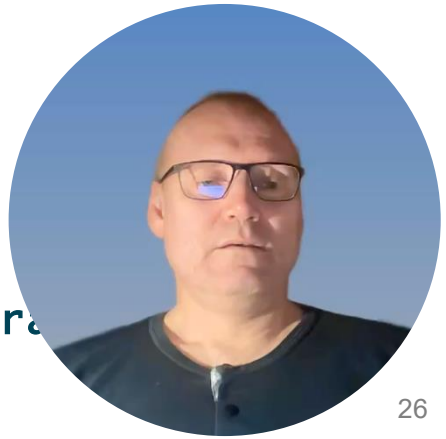
'93 HTML & Mosaic Browser (www)

'76 Multics Relational Data Store

'94 Amazon '98 Google

'04 Facebook '10 Instagra

'06 Twitter



NoSQL & BigData

Introduksjon til NoSQL databaser

Høgskulen på Vestlandet

- ✓ **NoSQL** er en sekkebetegnelse på en rekke nyere alternativer til relasjonsdatabaser.
  - NoSQL = **Not Only SQL**
  - Komplekse datastrukturer
  - Semistrukturerte data
  - Distribuerte løsninger
  - Løsere krav til transaksjonshåndtering

- ✓ **Big Data**: datasett som er så store og komplekse at de tøyer grensene for hva som er mulig å håndtere, både med hensyn til lagring, utveksling og behandling.
  - **Volume + Velocity + Variety + Veracity + Value**

Oppsummering

Fra objekter til NoSQL, Database History

Høgskulen på Vestlandet

Database Historie

- ✓ **1960-tallet**: hierarkiske databaser og nettverksdatabaser
- ✓ **1970-tallet:** Relasjonsdatabaser
- ✓ **1990-tallet:** Objekt-relasjonsdatabaser: Utvidelser av relasjonsdatabasene
- ✓ **2000-tallet/ 2010-tallet:**
  - Store selskaper og sosiale mediaer
  - NoSQL databasene gjør inn-tog.

✓ Databaser og objekter

- Array-typer
- Oppramstyper (Enumerated types)
- Sammensatte typer (Composite types)
- Intervalltyper (Range types)
- Geometrityper (PostGIS)

DEMO